

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Brainstorming & Ideation Phase

Project	PocketSmart AI
Documentation Phase	Brainstorming & Ideation Phase
Reference	PocketSmart AI project document

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Brainstorming & Ideation Phase

1. Introduction

PocketSmart AI was conceived to address a common planning problem: managing a fixed budget across lifestyle needs can become difficult because users must compare many products, services, platforms, prices, styles, and contextual requirements. The project document proposes a GenAI-powered, cross-platform recommendation system that accepts user preferences, budgets, and contextual inputs and converts them into curated recommendations.

1. Problem Identification

Budget planning for everyday lifestyle needs such as home decoration, parties, and jewelry shopping can involve many choices and price ranges.

- Users need recommendations that consider both what they want and what they can afford. The project therefore focuses on budget-aware, personalized recommendations instead of simple product listing.
- The project also identifies a need to support different input types. In the Jewelry Planner, an outfit image can be supplied so that the recommendation process can consider color coordination and aesthetics.

2. Project Idea

- The proposed solution is PocketSmart AI, an intelligent assistant that combines a web interface, a backend service, and Gemini 1.5 Flash Pro to generate contextual recommendations.
- The idea is organized around three planners: Home Interior Planning, Party Budget Planning, and Jewelry Recommendations for occasions.
- The recommendation experience is intended to connect users with options from platforms mentioned in the project document, including Amazon, Flipkart, IKEA, Swiggy, Zomato, and OYO.

3. Ideation Scenarios

- Home Interior Planning: users provide a home budget, room type, quantities, style, and related preferences. The AI recommends cost-effective furniture, decor, lighting, and other items while considering functionality, style, and price.
- Party Budget Planning: users provide total budget, guest count, event type, and venue details. The AI allocates the budget across catering, decoration, entertainment, and related services and provides suitable options.
- Jewelry Recommendations: users provide a budget, occasion, and style preferences and may upload an outfit image. The AI uses the supplied context to generate style-matched jewelry options.

4. Objectives

Build a single platform for budget-based recommendations across multiple lifestyle categories.

- Use Gemini 1.5 Flash Pro for budget interpretation, contextual reasoning, structured suggestions, and image-aware recommendation generation.

Provide a lightweight and user-friendly web experience through HTML, CSS, JavaScript, and Jinja2 templates.

Support personalized interaction through registration, login, sessions, saved queries, and recommendation history.

5. Proposed Technology Direction

- The project document identifies Flask/FastAPI as the backend technology direction across its architecture and implementation milestones, with the detailed implementation using FastAPI routes such as /generate-home, /generate-party, and /generate-jewelry.
- Gemini 1.5 Flash Pro forms the AI layer. Third-party platforms are described as sources for product or service information, with mock API calls or simulated scraping allowed during development.
- The frontend uses HTML/CSS and JavaScript, with Jinja2 templates used in the detailed UI milestone.

6. Expected Outcome

- The ideation phase leads to a modular assistant that can transform a user's budget and preferences into practical recommendations.
- The intended experience is budget-friendly, fast, personalized, and capable of handling multiple domains through separate planner modules.

7. Conclusion

The brainstorming and ideation phase establishes PocketSmart AI as a practical application of generative AI to everyday budget planning. The idea is grounded in three concrete user scenarios and provides the foundation for the requirement, design, planning, development, testing, documentation, and demonstration phases.

Phase Summary

Prepared for college project submission

Project	PocketSmart AI
Documentation Phase	Brainstorming & Ideation Phase
Reference	PocketSmart AI project document

Aspect	PocketSmart AI – Brainstorming & Ideation Phase
Project focus	Budget-aware, AI-powered recommendations across Home, Party, and Jewelry domains.
AI layer	Gemini 1.5 Flash Pro for contextual and multimodal recommendation processing.
Backend	FastAPI routes, authentication, sessions, recommendation handling, and startup configuration.
Frontend	HTML/CSS, JavaScript, and Jinja2 templates for inputs and structured recommendation display.
Main outcome	A modular, user-oriented recommendation workflow aligned with the documented project architecture.

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Requirement Analysis Phase

Project	PocketSmart AI
Documentation Phase	Requirement Analysis Phase
Reference	PocketSmart AI project document

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Requirement Analysis Phase

1. Introduction

The requirement analysis for PocketSmart AI converts the project idea into functional, technical, and user-facing requirements. The project document identifies the core technologies, planner inputs, AI capabilities, backend routes, authentication functions, session features, and testing expectations needed to implement the system.

1. Functional Requirements

- The system shall provide separate planner modules for Home Interior, Party, and Jewelry recommendations.
- The Home Planner shall accept budget, room details, quantities, style, and related preferences and generate recommendations for furniture, decor, lighting, and similar categories.
- The Party Planner shall accept total budget, guest count, event type, and venue details and generate suggestions covering catering, decoration, entertainment, and related services.
- The Jewelry Planner shall accept budget, occasion, and style preferences and support an optional outfit image for multimodal analysis.
- The system shall provide user registration, login, logout, secure token handling, session information, and session-specific data.
- The system shall maintain recommendation history so users can review or reuse previous recommendation queries and results.

2. AI Requirements

- Gemini 1.5 Flash Pro shall process text and numerical inputs and, where applicable, images.

Prompts shall be designed for budget interpretation, category segmentation, recommendation generation, and image-aware jewelry suggestions.

- The AI layer shall support structured recommendations and platform-aware product or service suggestions.

Fallback or default recommendations shall be available when the AI produces insufficient results.

3. Backend Requirements

- The backend shall expose planner routes such as /generate-home, /generate-party, and /generate-jewelry.
- Authentication routes include /register, /login, /logout, and /token.

Session-related routes include /session-info and /session-data.

- The implementation includes /recommendations-details for detailed recommendations, /history for previous recommendations, and /startup for initialization.

CORS and session handling shall support communication between the frontend and backend.

4. Frontend Requirements

- The frontend shall provide a home page and separate forms for the three planner modules.
- The forms shall collect the inputs relevant to each planning scenario.

AI responses shall be displayed in structured, card-like layouts containing product suggestions, item details, and prices where available.

- The interface shall include authentication pages, a user dashboard, planner pages, recommendation pages, and recommendation history.

5. Technical Prerequisites

- The source document lists cloud access and AI configuration, Python basics, and framework knowledge among the prerequisites.
- The project workflow specifically requires Google Cloud access or the relevant platform for Gemini API access, API-key configuration, usage limits, and multimodal testing.
- The backend requires the libraries and environment configuration needed for the FastAPI implementation and .env loading described in the project.

6. Non-Functional Requirements

- The application should be modular and maintainable, with recommendation logic and AI calls separated into utility/service components.
- The interface should be responsive and intuitive.
- The system should validate inputs and handle edge cases to reduce failures.
- The recommendation process should be tested for budget adherence, response quality, platform accuracy, and contextual awareness.

7. Requirements Traceability

User budget and preference requirements are addressed by the three planner modules.

AI requirements are addressed by Gemini 1.5 Flash Pro and the prompt orchestration layer.

Personalization requirements are addressed through authentication, sessions, dashboard data, and recommendation history.

Reliability requirements are addressed through validation, testing, prompt optimization, and fallback recommendations.

8. Conclusion

The requirement analysis establishes a clear boundary for PocketSmart AI. It connects user inputs, AI processing, backend routing, frontend presentation, authentication, and testing into one coherent system requirement set.

Phase Summary

Prepared for college project submission

Project	PocketSmart AI
Documentation Phase	Requirement Analysis Phase
Reference	PocketSmart AI project document

Aspect	PocketSmart AI – Requirement Analysis Phase
Project focus	Budget-aware, AI-powered recommendations across Home, Party, and Jewelry domains.
AI layer	Gemini 1.5 Flash Pro for contextual and multimodal recommendation processing.
Backend	FastAPI routes, authentication, sessions, recommendation handling, and startup configuration.
Frontend	HTML/CSS, JavaScript, and Jinja2 templates for inputs and structured recommendation display.
Main outcome	A modular, user-oriented recommendation workflow aligned with the documented project architecture.

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Design Phase

Project	PocketSmart AI
Documentation Phase	Project Design Phase
Reference	PocketSmart AI project document

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Design Phase

1. Introduction

The project design phase defines how PocketSmart AI is organized into interacting components. The project document describes a modular architecture in which the frontend collects inputs, the FastAPI backend handles routes and sessions, and the Gemini AI layer processes context and generates recommendations.

1. Architecture Overview

PocketSmart AI is designed as an intelligent, modular platform for budget-aware recommendations across Home, Party, and Jewelry domains.

- The architecture contains a frontend UI, FastAPI backend, Gemini AI layer, and connections to third-party product or service sources.
- The backend acts as the bridge between the frontend and the recommendation logic implemented through the AI utility/service layer.

2. Major Components

- Frontend UI: HTML/CSS, JavaScript, and Jinja2 templates collect budgets, preferences, event or room details, and optional images and display recommendations.
- FastAPI Backend: handles request routing, authentication, session control, structured inputs and outputs, and communication with the AI layer.
- Gemini AI Layer: uses Gemini 1.5 Flash Pro for budget interpretation, multimodal image analysis, prompt processing, and structured recommendation generation.
- External Platform Layer: represents sources such as Amazon, Flipkart, IKEA, Swiggy, Zomato, and OYO for product or service information.

3. Planner Design

- Home Planner design centers on room type, quantities, budget, style, and preferences. Output is organized around furniture, decor, lighting, and related home items.
- Party Planner design centers on event type, guest count, total budget, and venue information. Output covers catering, decoration, entertainment, and related services.
- Jewelry Planner design centers on budget, occasion, style, and optional outfit image. Output provides jewelry suggestions matched to the supplied context.

4. API Route Design

- /generate-home handles home-related recommendation requests.
- /generate-party handles event and party planning requests.
- /generate-jewelry supports text and optional image inputs for jewelry recommendations.
- /register, /login, /logout, and /token provide authentication functions.
- /session-info and /session-data support personalized session interaction.
- /recommendations-details and /history support detailed recommendations and past-query review.

- /startup initializes application services and configuration.

5. Modular Design

- The project document separates routing, recommendation logic, AI calls, and input/output structures. The earlier architecture description identifies routes/, services/, and models/ as modular areas.
- The detailed implementation uses a Gemini utility/service layer for prompt orchestration, budget formatting, domain segmentation, and image analysis.

This separation keeps planner logic and AI interaction organized while allowing the frontend to communicate through clear endpoints.

6. Data and Interaction Flow

- The user opens a planner and enters the required budget and contextual information.
- The frontend sends the planner request to the corresponding FastAPI route.
- The backend prepares the input and communicates with the Gemini recommendation layer.
- Gemini interprets the budget and context and generates domain-specific recommendations.
- The backend returns structured results to the frontend, where they are displayed in recommendation cards or pages.

Relevant session and history features support personalized review and reuse.

7. UI Design

- The design includes a main home page, authentication pages, user dashboard, three planner pages, three recommendation pages, and a recommendation history page.
- The recommendation interface uses structured card-like layouts for readable presentation of suggestions, details, and prices.

8. Conclusion

The design phase translates the PocketSmart AI concept into a modular technical architecture. Its separation of frontend, backend, AI processing, authentication, and recommendation modules provides a clear structure for implementation and testing.

Phase Summary

Prepared for college project submission

Project	PocketSmart AI
Documentation Phase	Project Design Phase
Reference	PocketSmart AI project document

Aspect	PocketSmart AI – Project Design Phase
Project focus	Budget-aware, AI-powered recommendations across Home, Party, and Jewelry domains.
AI layer	Gemini 1.5 Flash Pro for contextual and multimodal recommendation processing.

Aspect	PocketSmart AI – Project Design Phase
Backend	FastAPI routes, authentication, sessions, recommendation handling, and startup configuration.
Frontend	HTML/CSS, JavaScript, and Jinja2 templates for inputs and structured recommendation display.
Main outcome	A modular, user-oriented recommendation workflow aligned with the documented project architecture.

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Planning Phase

Project	PocketSmart AI
Documentation Phase	Project Planning Phase
Reference	PocketSmart AI project document

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Planning Phase

1. Introduction

The project planning phase organizes the PocketSmart AI work into a sequence of activities and milestones. The source project document defines a workflow beginning with Gemini initialization, followed by core functionality development, backend implementation, UI development, and testing and optimization.

1. Planning Objective

- The objective is to develop PocketSmart AI in manageable stages so that AI integration, planner logic, backend routing, frontend interaction, authentication, and testing are implemented systematically.
- The plan follows the project document's milestone-oriented workflow and keeps the three recommendation domains aligned with the same overall architecture.

2. Phase-Wise Work Plan

- Stage 1 – Gemini AI Initialization: establish authenticated Gemini API access, configure keys and limits, test text and image prompts, and refine prompts.
- Stage 2 – Core Functionality Development: build the backend structure, planner endpoints, Gemini prompts, and modular recommendation logic.
- Stage 3 – Backend Integration: implement FastAPI routes, authentication, session handling, CORS/static routing, startup logic, and recommendation/history endpoints.
- Stage 4 – UI Development: create planner forms, authentication pages, dashboard, recommendation displays, and Jinja2-based frontend integration.
- Stage 5 – Testing & Optimization: run real-world budget scenarios, evaluate recommendation quality, validate budget adherence and platform accuracy, and optimize prompts and validations.

3. Milestone 1 – Gemini AI Initialization

Set up Google Cloud access or the relevant platform and enable Gemini API access.

- Create and securely configure the API key.
- Test a sample Python API call for text-only and text-plus-image prompts.
- Refine prompts for budget interpretation and product recommendation generation.

4. Milestone 2 – Core Functionalities

Initialize the backend framework and required libraries.

Load environment configuration.

- Implement Home, Party, and Jewelry recommendation routes.
- Connect the routes to recommendation functions and the Gemini utility/service layer.
- Implement authentication and session-related routes.

5. Milestone 3 – Backend Integration

Define FastAPI routes for all planner and authentication functions.

- Implement structured input/output handling.

Configure CORS and static routing.

- Implement startup initialization and the uvicorn application entry point.
- Add recommendation details and history handling.

6. Milestone 4 – UI Development

- Create the home page and category selectors.
- Implement individual planner forms connected to FastAPI POST routes.
- Create structured recommendation displays.

Integrate authentication and session-based personalization.

- Implement pages for dashboard, planners, recommendations, and history.

7. Milestone 5 – Testing & Optimization

- Test home decor setups using varied budgets and preferences.
- Test party packages involving food, venue, and decoration requirements.
- Test jewelry suggestions using occasions, styles, budgets, and image inputs.
- Evaluate Gemini response quality, budget adherence, platform accuracy, and contextual awareness.
- Add input validation, refine prompts, and provide fallback recommendations when results are insufficient.

8. Deliverables by Stage

AI initialization produces a working Gemini integration.

Core functionality produces planner-specific recommendation logic.

Backend integration produces accessible and maintainable FastAPI routes.

UI development produces a complete user interaction layer.

Testing and optimization produces a validated recommendation workflow and improved handling of edge cases.

9. Conclusion

The planning phase provides a structured path from AI setup to a complete web application. The milestone sequence in the project document ensures that the major PocketSmart AI components are developed and validated in an organized order.

Phase Summary

Prepared for college project submission

Project	PocketSmart AI
Documentation Phase	Project Planning Phase
Reference	PocketSmart AI project document

Aspect	PocketSmart AI – Project Planning Phase
Project focus	Budget-aware, AI-powered recommendations across Home, Party, and Jewelry domains.
AI layer	Gemini 1.5 Flash Pro for contextual and multimodal recommendation processing.
Backend	FastAPI routes, authentication, sessions, recommendation handling, and startup configuration.
Frontend	HTML/CSS, JavaScript, and Jinja2 templates for inputs and structured recommendation display.
Main outcome	A modular, user-oriented recommendation workflow aligned with the documented project architecture.

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Development Phase

Project	PocketSmart AI
Documentation Phase	Project Development Phase
Reference	PocketSmart AI project document

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Development Phase

1. Introduction

The development phase implements the PocketSmart AI design and requirements through the backend, AI integration, planner modules, authentication, sessions, and frontend. The project document gives specific activities and routes that form the implementation basis.

1. Development Environment and AI Setup

- The development begins with Google Cloud or platform access for Gemini 1.5 Flash Pro, API-key creation, usage configuration, and connectivity validation.

A sample Python call is used to confirm text-only and multimodal communication before integrating AI into planner workflows.

2. Backend Initialization

- The project uses FastAPI in the detailed implementation milestone. Required libraries are imported and environment variables are loaded through the configured environment file.
- The backend is organized so that routing, recommendation logic, authentication, and session handling can be maintained separately.

3. Home Planner Development

- The /generate-home route handles home interior recommendation requests.
- The planner accepts home-related preferences and budget information and generates suggestions for furniture, decor, lighting, and related categories.
- The recommendation function connects the AI-generated context with platform references such as Amazon, Flipkart, and IKEA as described in the project.

4. Party Planner Development

- The /generate-party route processes party details and guest count.
- The recommendation workflow covers catering, venue, decoration, entertainment, and related options.
- The project identifies sources such as Swiggy, Zomato, and OYO for party-related food and accommodation/service options.

5. Jewelry Planner Development

- The /generate-jewelry route supports text inputs and optional image inputs.
- Users provide budget, occasion, and style information and may upload an outfit image.
- Gemini 1.5 Flash Pro processes the contextual information and image to produce style-matched jewelry recommendations, including options associated with Amazon and Flipkart.

6. Authentication and Session Development

- /register handles new user registration and secure credential storage as described in the source document.
- /login authenticates users and starts a session.

- /logout terminates the current session.
- /token issues a secure JWT token after successful authentication.
- /session-info and /session-data provide session and personalization information.

7. Recommendation and History Features

- /recommendations-details returns detailed AI-generated recommendations based on budget, preferences, and selected category.
- /history retrieves previous recommendation queries and results for review or reuse.
- The user dashboard presents recent recommendations, saved queries, and personalized insights.

8. Frontend Development

- The frontend is built with HTML/CSS, JavaScript, and Jinja2 templates.

Individual forms are created for Home, Party, and Jewelry planners and are connected to FastAPI POST routes.

AI responses are displayed using structured card-like layouts containing product suggestions, details, and prices.

- The interface includes the home page, testimonials page, register page, login page, dashboard, planner pages, recommendation pages, and history page.

9. Development Quality Measures

Prompt templates are tuned for each planner domain.

Input validation is added to reduce edge-case failures.

Fallback/default recommendations are included when AI output is insufficient.

CORS, sessions, and modular code organization support clean frontend-backend interaction.

10. Conclusion

The development phase turns the PocketSmart AI architecture into an operational web application. The implementation connects planner-specific inputs, Gemini-based recommendation logic, authentication, sessions, history, and the user interface into one system.

Phase Summary

Prepared for college project submission

Project	PocketSmart AI
Documentation Phase	Project Development Phase
Reference	PocketSmart AI project document

Aspect	PocketSmart AI – Project Development Phase
Project focus	Budget-aware, AI-powered recommendations across Home, Party, and Jewelry domains.
AI layer	Gemini 1.5 Flash Pro for contextual and multimodal recommendation processing.
Backend	FastAPI routes, authentication, sessions, recommendation handling, and startup configuration.

Aspect	PocketSmart AI – Project Development Phase
Frontend	HTML/CSS, JavaScript, and Jinja2 templates for inputs and structured recommendation display.
Main outcome	A modular, user-oriented recommendation workflow aligned with the documented project architecture.

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Testing Phase

Project	PocketSmart AI
Documentation Phase	Project Testing Phase
Reference	PocketSmart AI project document

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Testing Phase

1. Introduction

The testing phase validates whether PocketSmart AI behaves reliably across its three recommendation domains and whether the generated results remain relevant to the supplied budget and context. The project document specifically identifies real-world scenario testing, response-quality evaluation, budget adherence, platform accuracy, validation, prompt optimization, and fallback behavior.

1. Testing Objectives

Validate the recommendation workflow across Home, Party, and Jewelry planners.

- Check whether AI responses follow the supplied budget and contextual requirements.
- Evaluate response quality and contextual awareness.
- Check platform accuracy for the product/service sources referenced by the project.

Identify input edge cases and insufficient AI responses.

2. Home Planner Testing

- Use varied home decor budgets and preferences to test the Home Planner.
- Test room-related inputs such as Living Room, Kitchen, and Bedroom and quantities for items such as lights, ceiling fans, and dining tables.
- Verify that generated recommendations consider functionality, style, and price.
- Check that the recommendation output is displayed correctly on the Home Interior Recommendations page.

3. Party Planner Testing

- Use different total budgets, guest counts, event types, and venue details.
- Verify that the system produces suggestions for catering, decoration, entertainment, and venue-related needs.
- Test the project-described sources such as Swiggy, Zomato, and OYO where applicable.
- Check that the Party Budget Recommendations page presents the results in a clear structure.

4. Jewelry Planner Testing

- Test combinations of budget, occasion, and style preferences.
- Test the optional outfit-image workflow to verify multimodal input handling.
- Evaluate whether the generated jewelry suggestions consider outfit style, color coordination, aesthetics, occasion, and budget.
- Check that the Jewelry Budget Recommendations page presents the generated options correctly.

5. Authentication and Session Testing

- Verify registration flow and credential handling.
- Verify login authentication and session creation.
- Verify logout behavior and session termination.

- Check /token, /session-info, and /session-data behavior for authenticated interaction.
- Verify that dashboard and history information is associated with the user's session.

6. AI Response Testing

- Evaluate Gemini responses for relevance to the submitted planner context.
- Check budget adherence and whether the response remains within the user's defined financial boundary.
- Evaluate whether category-specific prompts return appropriate recommendation structures.
- Test insufficient-result cases and verify fallback/default recommendations.

7. UI and Integration Testing

- Verify that planner forms submit correctly to the corresponding FastAPI routes.
- Check that recommendation results render in structured cards and pages.
- Test navigation among home, authentication, dashboard, planners, recommendations, and history.
- Check responsive and user-friendly behavior across the frontend described in the project.

8. Optimization Activities

- Refine prompts when responses do not sufficiently match the user's budget or context.
- Add validation for edge cases in planner inputs.

Improve session handling where required.

- Use fallback/default recommendations when AI results are insufficient.

9. Expected Testing Result

- Successful testing should demonstrate a consistent end-to-end flow from user input to AI recommendation and frontend display across the three planner domains.
- The project document defines reliability, accuracy, and contextual awareness as the central goals of this validation stage.

10. Conclusion

The testing phase provides evidence that the developed PocketSmart AI workflow can be exercised through realistic budget scenarios. Continuous prompt refinement, input validation, and fallback handling are used to improve reliability and recommendation quality.

Phase Summary

Prepared for college project submission

Project	PocketSmart AI
Documentation Phase	Project Testing Phase
Reference	PocketSmart AI project document

Aspect	PocketSmart AI – Project Testing Phase
Project focus	Budget-aware, AI-powered recommendations across Home, Party, and Jewelry domains.
AI layer	Gemini 1.5 Flash Pro for contextual and multimodal recommendation processing.
Backend	FastAPI routes, authentication, sessions, recommendation handling, and startup configuration.
Frontend	HTML/CSS, JavaScript, and Jinja2 templates for inputs and structured recommendation display.
Main outcome	A modular, user-oriented recommendation workflow aligned with the documented project architecture.

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Documentation Phase

Project	PocketSmart AI
Documentation Phase	Project Documentation Phase
Reference	PocketSmart AI project document

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Documentation Phase

1. Introduction

The documentation phase records the PocketSmart AI system in a structured form so that its purpose, architecture, workflow, implementation, user interface, routes, testing activities, and final outcome can be understood and presented consistently. This documentation is based on the technical and functional material contained in the project document.

1. Project Overview

PocketSmart AI is a GenAI-powered, cross-platform recommendation system for budget-aware lifestyle planning.

It covers Home Interior Planning, Party Budget Planning, and Jewelry Recommendations and uses Gemini 1.5 Flash Pro for contextual and multimodal recommendation generation.

2. Core Technology Documentation

- Gemini 1.5 Flash Pro is documented as the AI layer for budget interpretation, image analysis, structured suggestions, and platform-aware recommendations.
- FastAPI is documented as the detailed backend framework used for routing, authentication, session control, and communication with the AI layer.

HTML/CSS, JavaScript, and Jinja2 templates form the frontend layer.

- The project references third-party platforms including Amazon, Flipkart, IKEA, Swiggy, Zomato, and OYO as product/service sources.

3. Architecture Documentation

- The documented architecture consists of frontend UI, backend API, Gemini AI processing, and third-party source integration.
- The source project also describes modular areas for routes, services/recommendation logic, and models/input-output schemas.

4. Route Documentation

- Planner routes: /generate-home, /generate-party, /generate-jewelry.
- Authentication routes: /register, /login, /logout, /token.

Session routes: /session-info and /session-data.

- Recommendation and history routes: /recommendations-details and /history.

Startup handling: /startup and the application __main__ entry point.

5. User Interface Documentation

- Home Page: introduces PocketSmart AI and provides access to planner modules and user actions.

Testimonials Page: presents user reviews and success stories.

Register and Login Pages: support account creation and authentication.

User Dashboard: presents recent recommendations, saved queries, and personalized insights.

- Planner Pages: collect inputs for Home, Party, and Jewelry recommendations.
- Recommendation Pages: present AI-generated results for each planner.
- Recommendation History Page: displays previous queries and results for review or reuse.

6. Implementation Documentation

- The project records Gemini setup, API-key configuration, connectivity testing, backend initialization, planner routing, authentication, session control, UI development, and testing/optimization as major implementation activities.
- The Gemini utility/service layer is responsible for prompt orchestration, budget formatting, domain segmentation, and image analysis in the detailed implementation description.

7. Testing Documentation

- The documented testing scope covers real-world budget inputs across Home, Party, and Jewelry domains.

Testing evaluates Gemini response quality, budget adherence, platform accuracy, contextual awareness, validation, and fallback recommendations.

8. User Workflow Documentation

A user enters or authenticates into the platform, chooses a planner, supplies the required budget and contextual information, and submits the request.

- The corresponding FastAPI route communicates with the Gemini recommendation layer.

Generated recommendations are displayed in the relevant recommendation page and can be associated with the user's session/history.

9. Documentation Standards for Submission

Each major phase should describe its objective, activities, technical implementation, outputs, and conclusion.

Terminology should remain consistent with the project: Home Planner, Party Planner, Jewelry Planner, Gemini 1.5 Flash Pro, FastAPI, recommendation history, sessions, and the documented routes.

Screenshots, code excerpts, route descriptions, and UI descriptions should be placed under the phase where they support the implementation or demonstration.

10. Conclusion

The documentation phase turns the technical work of PocketSmart AI into a structured submission record. It preserves the project's architecture, implementation decisions, planner workflows, interface structure, and testing approach in a form suitable for review and presentation.

Phase Summary

Prepared for college project submission

Project	PocketSmart AI
Documentation Phase	Project Documentation Phase
Reference	PocketSmart AI project document

Aspect	PocketSmart AI – Project Documentation Phase
Project focus	Budget-aware, AI-powered recommendations across Home, Party, and Jewelry domains.
AI layer	Gemini 1.5 Flash Pro for contextual and multimodal recommendation processing.
Backend	FastAPI routes, authentication, sessions, recommendation handling, and startup configuration.
Frontend	HTML/CSS, JavaScript, and Jinja2 templates for inputs and structured recommendation display.
Main outcome	A modular, user-oriented recommendation workflow aligned with the documented project architecture.

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Demonstration Phase

Project	PocketSmart AI
Documentation Phase	Project Demonstration Phase
Reference	PocketSmart AI project document

PROJECT DOCUMENTATION

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Demonstration Phase

1. Introduction

The demonstration phase presents the completed PocketSmart AI system through its main user workflows. The project document describes the application pages, planner modules, recommendation outputs, authentication flow, dashboard, and history features that can be shown during a project demonstration.

1. Demonstration Objective

Demonstrate that PocketSmart AI can accept budget and contextual inputs and return AI-generated recommendations across Home, Party, and Jewelry domains.

- Show the relationship between the frontend interface, FastAPI routes, Gemini recommendation layer, authentication/session features, and recommendation history.

2. Demonstration Flow – Application Entry

- Begin with the PocketSmart AI home page and explain that it provides access to the platform's main features and planner modules.
- Show the available user actions and the navigation to registration or login where required.

3. Demonstration Flow – Registration and Login

- Show the Register Page for creating a user account.
- Show the Login Page for authenticating an existing user.
- Explain that authenticated interaction supports dashboard, session, and saved recommendation functionality.

Demonstrate logout to show session termination.

4. Demonstration Flow – Home Interior Planner

- Open the Home Interior Budget Planner.
- Enter a sample budget, room type, style, and required quantities based on the project scenario.
- Submit the request through the Home Planner flow.
- Display the Home Interior Recommendations Page and explain how the generated suggestions are matched to budget and style preferences.
- Mention the project-described sources such as IKEA and Amazon.

5. Demonstration Flow – Party Planner

- Open the Party Budget Planner.
- Enter total budget, guest count, event type, and venue details.
- Submit the request and display the Party Budget Recommendations Page.
- Explain the suggested food, venue, decoration, and related options and the project references to Swiggy, Zomato, and OYO.

6. Demonstration Flow – Jewelry Planner

- Open the Jewelry Budget Planner.
- Enter budget, occasion, and style preferences.

Optionally upload an outfit image to demonstrate multimodal input.

- Display the Jewelry Budget Recommendations Page and explain that the recommendations use occasion, outfit context, style, and budget.
- Mention the project-described product sources such as Amazon and Flipkart.

7. Demonstration of Dashboard and History

- Open the User Dashboard to show recent recommendations, saved queries, and personalized insights.
- Open the Recommendation History Page to demonstrate that past recommendation queries and results can be reviewed or reused.

8. Technical Demonstration Points

- Explain the frontend-to-backend request flow.

Identify the relevant FastAPI route for the selected planner.

- Explain that Gemini 1.5 Flash Pro processes the planner context and generates recommendations.
- Show that structured results are rendered on the corresponding recommendation page.

If appropriate for the project environment, demonstrate validation and fallback behavior when inputs are incomplete or AI output is insufficient.

9. Demonstration Checklist

- Home page and navigation are available.

Registration/login flow is ready.

- Home Planner accepts budget and room-related inputs.
- Party Planner accepts budget, guest, event, and venue inputs.
- Jewelry Planner accepts text inputs and optional image input.
- Recommendation pages display generated results.

Dashboard and history pages display saved/recent information.

- The main AI and backend workflow can be explained clearly.

10. Conclusion

The demonstration phase presents PocketSmart AI as an end-to-end GenAI application. By walking through authentication, the three planners, recommendation results, dashboard, and history, the demonstration shows how the documented architecture and development activities become a user-facing system.

Phase Summary

Prepared for college project submission

Project	PocketSmart AI
Documentation Phase	Project Demonstration Phase

Project	PocketSmart AI
Reference	PocketSmart AI project document

Aspect	PocketSmart AI – Project Demonstration Phase
Project focus	Budget-aware, AI-powered recommendations across Home, Party, and Jewelry domains.
AI layer	Gemini 1.5 Flash Pro for contextual and multimodal recommendation processing.
Backend	FastAPI routes, authentication, sessions, recommendation handling, and startup configuration.
Frontend	HTML/CSS, JavaScript, and Jinja2 templates for inputs and structured recommendation display.
Main outcome	A modular, user-oriented recommendation workflow aligned with the documented project architecture.